

View e rota da tela de edição de usuário

Função de view, formulário e rota que servem a tela de edição.

O que a view faz

A view `usuario_editar` atende a mesma URL em dois métodos:

Método	Resposta	Comportamento
GET	200	Monta o formulário já preenchido com os dados do usuário e renderiza a tela.
POST válido	302	Grava as alterações, registra a mensagem “Dados atualizados.” e redireciona para a própria tela.
POST inválido	200	Devolve a tela com os valores digitados e a mensagem de erro embaixo de cada campo.

O usuário é buscado com `get_object_or_404`: um id inexistente devolve 404 em vez de estourar exceção. Depois de salvar, a view redireciona em vez de renderizar — assim atualizar a página no navegador não reenvia o formulário.

Rota

URL	Nome	View
<code>/usuarios/<int:pk>/editar/</code>	<code>usuarios:editar</code>	<code>usuario_editar</code>

As rotas ficam em `usuarios/urls.py` com `app_name = "usuarios"`, então os templates referenciam a tela por nome — `{% url 'usuarios:editar' usuario.pk %}` — e não por caminho escrito à mão.

Código — view

`usuarios/views.py`

```
"""Views do app de usuários – FioFora."""
from django.contrib import messages
from django.shortcuts import get_object_or_404, redirect, render

from .forms import UsuarioForm
from .models import Usuario


def usuario_editar(request, pk):
    """Tela de edição dos dados de um usuário."""
    usuario = get_object_or_404(Usuario, pk=pk)

    if request.method == "POST":
        form = UsuarioForm(request.POST, instance=usuario)
        if form.is_valid():
            form.save()
            messages.success(request, "Dados atualizados.")
            return redirect("usuarios:editar", pk=usuario.pk)
    else:
        form = UsuarioForm(instance=usuario)
```

```

    return render(
        request,
        "usuarios/usuario_editar.html",
        {
            "usuario": usuario,
            "form": form,
            "secao_ativa": "perfil",
        },
    )

```

Código — rota

usuarios/urls.py

```

"""Rotas do app de usuários – FioFora.

Ligar ao projeto em fiofora/urls.py:

    path("usuarios/", include("usuarios.urls")),
"""
from django.urls import path

from . import views

app_name = "usuarios"

urlpatterns = [
    path("<int:pk>/editar/", views.usuario_editar, name="editar"),
    path("<int:pk>/remover/", views.usuario_remove, name="remover"),
]

```

Código — formulário

A lista GRUPOS define quais campos aparecem e em qual cartão; o método grupos () entrega isso pronto para o template.

usuarios/forms.py

```

"""Formulários do app de usuários – FioFora."""
from django import forms

from .models import Usuario

# Cada tupla vira um cartão na tela de edição: (título, texto de apoio, campos).
GRUPOS = [
    (
        "Dados pessoais",
        "Essas informações não aparecem nos seus anúncios.",
        ["nome", "email", "telefone", "data_nascimento"],
    ),
    (
        "Localização",
        "Usada para estimar prazo e disponibilidade de entrega.",
        ["cidade", "estado", "cep"],
    ),
]

# Campos que ocupam a linha inteira; os demais ficam dois por linha.
CAMPOS_LARGOS = []

class UsuarioForm(forms.ModelForm):
    """Formulário da tela de edição de usuário, organizado em cartões."""

```

```

class Meta:
    model = Usuario
    fields = [campo for _, _, campos in GRUPOS for campo in campos]
    widgets = {
        "data_nascimento": forms.DateInput(attrs={"type": "date"}, format="%Y-%m-%d"),
        "telefone": forms.TextInput(attrs={"placeholder": "(00) 00000-0000"}),
        "cep": forms.TextInput(attrs={"placeholder": "00000-000"}),
        "estado": forms.TextInput(attrs={"placeholder": "RJ", "maxlength": 2}),
        "email": forms.EmailInput(attrs={"placeholder": "voce@exemplo.com"}),
    }
    help_texts = {
        "email": "É com este e-mail que você entra na sua conta.",
    }

def grupos(self):
    """Devolve os campos agrupados, para o template montar os cartões."""
    for titulo, apoio, nomes in GRUPOS:
        campos = [
            {"campo": self[nome], "largo": nome in CAMPOS_LARGOS}
            for nome in nomes
            if nome in self.fields
        ]
        if campos:
            yield {"titulo": titulo, "apoio": apoio, "campos": campos}

```